



TRƯỜNG ĐẠI HỌC

SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

HCMC University of Technology and Education

Bài 7: Tuple-Dictionary





Nội dung:

- Tuple
- Dictionary

1. Tuple

- Tuple là một kiểu dữ liệu trong Python, tương tự như **list** nhưng có tính bất biến (immutable).
- Một khi đã tạo, **không thể thay đổi giá trị** của các phần tử trong tuple.
- Tuple có thể chứa nhiều kiểu dữ liệu khác nhau như số, chuỗi, list, hoặc thậm chí là tuple khác

a. Cách khai báo 1 Tuple

```
numbers = (1, 2, 3, 4, 5)
print(numbers)
```

```
fruits = "Táo", "Cam", "Xoài"
print(fruits)
```

```
letters = tuple(["a", "b", "c"])
print(letters)
```

```
single_element = (10,)
print(single_element) # Output: (10,)
```

b. Truy cập phần tử trong Tuple

```
colors = ("red", "green", "blue")  
  
print(colors[0]) # Output: red  
print(colors[1]) # Output: green  
print(colors[-1]) # Output: blue
```

c. Cắt (slicing) tuple

```
numbers = (10, 20, 30, 40, 50, 60)  
  
print(numbers[1:4]) # Output: (20, 30, 40)  
print(numbers[:3]) # Output: (10, 20, 30)  
print(numbers[-3:]) # Output: (40, 50, 60)
```

d. Tính bất biến của Tuple

```
colors = ("red", "green", "blue")
```

```
colors[1] = "black" # ❌ Error!
```

```
colors = ("red", "green", "blue")
```

```
colors[1] = "black" # ❌ Error!
```

```
temp = list(colors)
```

```
temp[1] = "black"
```

```
colors = tuple(temp)
```

```
print(colors) # Output: ('red', 'black', 'blue')
```

e. Duyệt qua Tuple

```
animals = ("dog", "cat", "tiger")  
  
for animal in animals:  
    print(animal)
```


f. Một số phương thức quan trọng

Phương thức	Mô tả	Ví dụ
<code>count(x)</code>	Đếm số lần phần tử <code>x</code> xuất hiện trong tuple.	<code>t = (1, 2, 2, 3, 2); t.count(2) → 3</code>
<code>index(x)</code>	Trả về chỉ mục (index) của phần tử <code>x</code> xuất hiện đầu tiên.	<code>t = (4, 5, 6, 7); t.index(6) → 2</code>
<code>len(tuple)</code>	Trả về độ dài của tuple (số phần tử).	<code>len((10, 20, 30)) → 3</code>
<code>max(tuple)</code>	Trả về giá trị lớn nhất trong tuple.	<code>max((3, 9, 1, 5)) → 9</code>
<code>min(tuple)</code>	Trả về giá trị nhỏ nhất trong tuple.	<code>min((3, 9, 1, 5)) → 1</code>
<code>sum(tuple)</code>	Tính tổng các phần tử (chỉ áp dụng với số).	<code>sum((1, 2, 3)) → 6</code>
<code>sorted(tuple)</code>	Sắp xếp tuple theo thứ tự tăng dần (trả về list).	<code>sorted((5, 1, 3)) → [1, 3, 5]</code>
<code>tuple(list)</code>	Chuyển list thành tuple.	<code>tuple([1, 2, 3]) → (1, 2, 3)</code>
<code>tuple(string)</code>	Chuyển chuỗi thành tuple (từng ký tự là phần tử).	<code>tuple("abc") → ('a', 'b', 'c')</code>

f. Một số phương thức quan trọng

```
numbers = (1, 2, 3, 2, 2, 4)
print(numbers.count(2)) # Output: 3
```

```
colors = ("red", "green", "blue", "green")
print(colors.index("green")) # Output: 1
```

f. Một số phương thức quan trọng

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
result = tuple1 + tuple2
print(result) # Output: (1, 2, 3, 4, 5, 6)
```

```
numbers = (1, 2, 3)
print(numbers * 3) # Output: (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

```
colors = ("red", "green", "blue", "green")
print("green" in colors) # Output: True
print("black" in colors) # Output: False
```

g. Tuple lồng nhau

```
nested_tuple = ((1, 2, 3), ("a", "b", "c"), (True, False))  
print(nested_tuple[1][2]) # Output: c
```

h. Tuple và List

📌 Khi nào dùng Tuple, khi nào dùng List?

Tiêu chí	List	Tuple
Tính thay đổi	Có thể thay đổi	Không thể thay đổi
Tốc độ	Chậm hơn	Nhanh hơn
Dùng trong	Khi cần thay đổi dữ liệu	Khi dữ liệu cố định
Bảo mật	Ít bảo mật hơn	Bảo mật hơn

Dùng tuple để trả về nhiều giá trị trong một hàm:

```
def get_info():  
    return "Duong Minh Thien", 31, "Automation Control"  
  
info = get_info()  
print(info)  # Output: ('Duong Minh Thien', 31, 'Automation Control')
```

Exercise 1:

- a) Tạo một tuple chứa 5 số nguyên. Viết chương trình tính tổng và giá trị lớn nhất của các số trong tuple.
- b) Viết chương trình nhập vào một danh sách tên, lưu dưới dạng tuple và hiển thị theo thứ tự bảng chữ cái.
- c) Viết chương trình nhập danh sách điểm số, tìm điểm cao nhất và số lần điểm đó xuất hiện

Exercise 1.a:

```
numbers = (10, 25, 7, 42, 18)
```

```
sum_numbers = sum(numbers)
```

```
max_number = max(numbers)
```

```
print(f"Tổng của các số trong tuple: {sum_numbers}")  
print(f"Giá trị lớn nhất trong tuple: {max_number}")
```


Exercise 1.a:

```
def calculate_sum_and_max(numbers):  
    total = sum(numbers)  
    maximum = max(numbers)  
    return total, maximum  
  
tuple_numbers = (12, 45, 67, 23, 89)  
  
total, maximum = calculate_sum_and_max(tuple_numbers)  
  
print(f"Tổng các số trong tuple: {total}")  
print(f"Giá trị lớn nhất trong tuple: {maximum}")
```

Exercise 1.b:

```
name_input = input("Nhập danh sách tên, cách nhau bởi dấu phẩy: ")  
  
name_list = name_input.split(",")  
  
name_tuple = tuple(sorted(name_list))  
  
print("Danh sách tên sau khi sắp xếp:", name_tuple)
```

Exercise 1.c:

```
scores_input = input("Nhập danh sách điểm, cách nhau bởi dấu cách: ")

scores = tuple(map(int, scores_input.split()))

max_score = max(scores)

count_max_score = scores.count(max_score)

print(f"Điểm cao nhất: {max_score}")
print(f"Số lần xuất hiện của điểm {max_score}: {count_max_score}")
```

2. Dictionary

- Dictionary trong Python là một cấu trúc dữ liệu dạng **key-value** (cặp khóa - giá trị).
- Dictionary không có thứ tự (unordered) và có thể thay đổi được (mutable).
- Key (khóa) phải là duy nhất, không thể trùng lặp.
- Value (giá trị) có thể là bất kỳ kiểu dữ liệu nào: số, chuỗi, danh sách, thậm chí là một dictionary khác.

a. Cách khai báo Dictionary

Cách 1: Sử dụng dấu ngoặc nhọn {}

```
student = {  
    "name": "Duong Minh Thien",  
    "age": 31,  
    "major": "Computer Science"  
}
```

Cách 2: Dùng hàm dict()

```
student2 = dict(name="Le Thi B", age=22, major="Mathematics")
```

In kết quả:

```
print(student)  
print(student2)
```

b. Cách truy xuất Dictionary

Có 2 cách để truy xuất giá trị từ dictionary:

```
print(student["name"]) # Output: Duong Minh Thien  
print(student.get("age")) # Output: 31
```

Sự khác biệt giữa [] và get:

- [] **báo lỗi** nếu khóa không tồn tại.
- .get() trả về **None** nếu khóa không tồn tại

```
print(student["gender"]) # KeyError  
print(student.get("gender")) # Output: None
```

c. Thêm hoặc cập nhật giá trị

```
student["age"] = 32 # cap nhat value  
student["email"] = "minhthien@hcmute.edu.vn" # Them key  
print(student)
```


d. Xóa phần tử trong Dictionary

Có 2 cách để xóa phần tử trong Dictionary:

Cách 1:

```
del student["email"]  
print(student)
```

Cách 2:

```
age = student.pop("age")  
print(f"Age has been deleted: {age}")  
print(student)
```

Lưu ý:

- del chỉ xóa phần tử.
- pop() vừa xóa vừa trả về giá trị.

e. Duyệt qua Dictionary

Duyệt qua các khóa:

```
for key in student.keys():  
    print(key)
```

Duyệt qua các giá trị:

```
for value in student.values():  
    print(value)
```

Duyệt qua các khóa và giá trị:

```
for key, value in student.items():  
    print(f"{key}: {value}")
```

f. Kiểm tra sự tồn tại của khóa

Dùng `in` để kiểm tra xem một khóa có tồn tại trong dictionary không:

```
if "age" in student:  
    print("Co key 'age' trong dictionary")
```

g. Dictionary lồng nhau

```
students = {  
    "student1": {"name": "Duong Minh Thien", "age": 31},  
    "student2": {"name": "Cristiano Ronaldo", "age": 40}  
}  
  
print(students["student1"]["name"]) # Output: Duong Minh Thien
```

h. Một số phương thức quan trọng của Dictionary

Phương thức	Chức năng
<code>dict.keys()</code>	Trả về danh sách các khóa
<code>dict.values()</code>	Trả về danh sách các giá trị
<code>dict.items()</code>	Trả về danh sách các cặp <code>(key, value)</code>
<code>dict.get(key, default)</code>	Lấy giá trị của <code>key</code> , nếu không có thì trả về <code>default</code>
<code>dict.update(dict2)</code>	Thêm hoặc cập nhật dictionary
<code>dict.pop(key)</code>	Xóa và trả về giá trị của <code>key</code>
<code>dict.clear()</code>	Xóa tất cả các phần tử

Viết đoạn chương trình khai báo **1 Dictionary** chứa **2 Dictionary** là thông tin của **2 bạn sinh viên** (tên, tuổi, ngành học). Sau đó thêm một bạn sinh viên mới vào Dictionary ở trên. Cuối cùng in ra danh sách thông tin của **3 bạn sinh viên**.

```
students = {  
    "SV001": {"name": "Duong Minh Thien", "age": 31, "major": "Automatic Control"},  
    "SV002": {"name": "Cristiano Ronaldo", "age": 40, "major": "Football"}  
}  
  
students["SV003"] = {"name": "Rafael Nadal", "age": 38, "major": "Tennis"}  
  
for student_id, info in students.items():  
    print(f"ID: {student_id}, Name: {info['name']}, Age: {info['age']}, Major: {info['major']}")
```

i. Tuple Làm khóa cho Dictionary:

```
location = {  
    (21.0285, 105.8542): "Hà Nội",  
    (10.7769, 106.7009): "TP Hồ Chí Minh"  
}  
print(location[(21.0285, 105.8542)]) # Output: Hà Nội
```

Exercise 2:

- a. Viết chương trình nhập từ bàn phím tên, tuổi và điểm trung bình của một sinh viên. Lưu vào dictionary và in ra.
- b. Viết chương trình quản lý danh bạ điện thoại sử dụng dictionary, cho phép:
 - Thêm một liên hệ mới.
 - Tìm kiếm số điện thoại theo tên.
 - Xóa một liên hệ theo tên.
- c. Viết chương trình nhập danh sách sinh viên và điểm số của họ, lưu vào dictionary, sau đó tính điểm trung bình của cả lớp



Homework:

Hoàn chỉnh **Excercise 1** và **Excercise 2**. Nộp lên trang **utexlms**

Lưu ý:

- Sinh viên viết chương trình và chú thích ý nghĩa câu lệnh bên cạnh, vd **# tăng biến i lên 1 đơn vị.**
- Sinh viên viết chương trình và lưu riêng lẻ từng file .py ứng với mỗi câu hỏi, vd cau1.py. Sau đó, nén 3 file .py thành file zip/rar và đặt tên theo quy định: HoTen_MSSV_BTChuong6 và nộp lên trang utexlms
- Thời gian làm bài 1 tuần kể từ ngày giao bài tập.
- Các chương trình giống nhau với tỷ lệ bất thường sẽ không được chấm điểm
- SV nộp trễ với bất kỳ lý do gì cũng không được giải quyết
- **SV PHẢI NỘP KÈM CÁC FILE BT TRÊN LỚP CÙNG VỚI BT VỀ NHÀ LUÔN NHÉ!**



TRƯỜNG ĐẠI HỌC

SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

HCMC University of Technology and Education

Thank you for
your listening

Q & A

